SMART RESUME ANALYZER

with AI-Based Feedback

SkillOrbit AI/ML Capstone Project Report

A beginner-level web application prototype for resume structure checks, role-keyword comparison, feedback and report download.

Prepared as an educational prototype

Abstract

The Smart Resume Analyzer accepts a PDF or DOCX resume, extracts readable text, checks for selected sections, compares the text against predefined role keywords, and displays improvement suggestions. It also provides a downloadable text report. The current prototype uses rule-based logic; it does not train a machine-learning model.

Problem Statement

Applicants may overlook missing resume sections or relevant terms for a target role. This project provides a quick first-pass checklist to help users review a resume.

Objectives

Accept PDF/DOCX files; extract readable text; check five basic resume indicators; compare text with role-related keywords; display suggestions; and provide a downloadable report.

Technology Stack

Python, Streamlit, PyPDF2, python-docx and regular expressions.

System Workflow

Upload resume → extract text → check resume sections → choose target role → compare keywords → show feedback → download report.

Module 1 — Upload and Parsing

The application accepts PDF and DOCX files and extracts text from their readable content. Scanned image-only PDFs may require OCR, which is outside the current scope.

Module 2 — Structure Score

Five indicators are checked: Education, Skills, Projects, Contact Information and Certifications. Each detected indicator contributes 20 points, producing a score from 0 to 100.

Module 3 — Keyword Checker

The user selects Data Analyst, Web Developer, AI Engineer or Cloud Engineer. The app compares resume text against a predefined keyword list and reports matched and missing terms. The percentage is an indicative keyword match, not an official ATS score.

Module 4 — Smart Feedback

Suggestions identify potentially missing sections, short resume content, contact details to verify, and role keywords to review. Users should add only accurate skills and experience.

Module 5 — Report Generation

The user can download a text report with the selected role, structure score, keyword match, matched/missing terms, section checks and suggestions.

Testing and Results

The project author tested the app by uploading a readable PDF, selecting a target role, reviewing the generated results and downloading the report. DOCX handling should be verified on the machine used for the final

demonstration.

Limitations

Checks rely on simple text matching and may not understand formatting or context. Scanned PDFs are not OCR-processed. The project does not store data in a database, train a model, or guarantee ATS/recruitment outcomes.

Future Scope

Possible improvements include OCR, job-description-specific matching, better section detection, a graphical dashboard, PDF report export, automated tests and carefully evaluated NLP features.

Conclusion

The project demonstrates a complete basic workflow for uploading a resume, extracting text, checking sections, comparing role keywords, presenting suggestions and downloading a report. It is a foundation for further development.

References

Streamlit documentation — https://docs.streamlit.io/ ; PyPDF2 documentation — https://pypdf2.readthedocs.io/ ; python-docx documentation — https://python-docx.readthedocs.io/ ; Python documentation — https://docs.python.org/3/.